

53.最大子数组和

1.问题

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium (56.00%)	7015	0	-	-	0

- ▼ Tags
- ▼ Companies

给你一个整数数组 `nums`，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

子数组 是数组中的一个连续部分。

示例 1:

```
输入: nums = [-2,1,-3,4,-1,2,1,-5,4]
输出: 6
解释: 连续子数组 [4,-1,2,1] 的和最大，为 6。
```

示例 2:

```
输入: nums = [1]
输出: 1
```

示例 3:

```
输入: nums = [5,4,-1,7,8]
输出: 23
```

提示:

- `1 <= nums.length <= 105`
- `104 <= nums[i] <= 104`

进阶: 如果你已经实现复杂度为 $O(n)$ 的解法，尝试使用更为精妙的 **分治法** 求解。

2.方法

好的，我们假设你之前完全没有接触过这个问题或类似的问题，一步一步来分解“最大子数组和”。

1. 目标是什么？

想象一下你有一串数字，有些是正数，有些是负数，比如 `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`。
我们的目标是，从这串数字里找出一个
连续的部分（一个“块”），让这个块里的所有数字加起来的和是所有可能块中最大的。

- 什么是“块”(子数组)？它必须是**连续的**。你不能跳过数字。
 - `[1, -3, 4]` 是一个有效的块。
 - `[4, -1, 2, 1]` 也是一个有效的块。
 - `[-2, 4, 1]` 不是一个有效的块，因为你跳过了 `1` 和 `3`。
- 什么是“和”？就是把你选中的那个块里的所有数字加起来。

- `[1, -3, 4]` 的和是 $1 + (-3) + 4 = 2$ 。
- `[4, -1, 2, 1]` 的和是 $4 + (-1) + 2 + 1 = 6$ 。
- 什么是“最大”？ 我们需要查看所有可能的连续块，计算它们的和，然后找出这些和里面最大的那个。

2. 为什么这个问题有点绕？（显而易见的方法不太好用）

- 把所有数字加起来就行了？ 不行，因为负数可能会让总和变小。我们可能只选列表的一部分就能得到更大的和。对于 `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`，所有数字加起来是 3，但我们看到块 `[4, -1, 2, 1]` 的和是 6，比 3 要大。
- 尝试每一个可能的块？ 你可以这么做！
 - 从第一个数 (-2) 开始：块有 `[-2]`，`[-2, 1]`，`[-2, 1, -3]`，... 一直到列表末尾。
 - 从第二个数 (1) 开始：块有 `[1]`，`[1, -3]`，`[1, -3, 4]`，... 一直到列表末尾。
 - ...依此类推。
 - 计算每个块的和，并记录下你见过的最大的和。
 - 问题：如果列表很长（比如有 10 万个数字），这样做会非常非常慢。计算机虽然快，但也没那么快。我们需要一个更聪明的方法。

3. 聪明的方法：Kadane 算法（逐步思考）

我们不一次性看所有的块，而是沿着列表一次只看一个数字，并且进行聪明的思考。

想象你正沿着列表走：`[-2, 1, -3, 4, -1, 2, 1, -5, 4]`

在走的过程中，我们需要两个关键信息：

1. `max_so_far`（到目前为止的最大和）：这是我们到目前为止，在任意地方见过的任何块的最大和。这将是我们的最终答案。
2. `max_ending_here`（以当前位置结尾的最大和）：这是恰好在咱们当前看的这个数字结束的块的最大和。这是最巧妙的部分！

让我们走一遍：

- 从第一个数字开始：-2
 - 唯一在这里结束的块是 `[-2]`。它的和是 -2。
 - `max_ending_here` = -2
 - `max_so_far` = -2（这是我们目前见过的最好的）
- 移动到第二个数字：1
 - 现在，在这里（在 1 这个位置）结束的最佳块是什么？我们有两个选择：
 1. 开始一个新块：`[1]`。和 = 1。
 2. 把前面那个在 -2 结束的最佳块扩展一下：`[-2, 1]`。和 = $-2 + 1 = -1$ 。
 - 哪个更好，1 还是 -1？显然 1 更好。所以，在这里结束的最佳块就是 `[1]`。
 - `max_ending_here` = 1
 - 现在，更新 `max_so_far`。新的 `max_ending_here` (1) 比旧的 `max_so_far` (-2) 大吗？是的。
 - `max_so_far` = 1
- 移动到第三个数字：-3
 - 在这里结束的最佳块？选择：
 1. 开始新的：`[-3]`。和 = -3。
 2. 扩展前面那个在这里结束的最佳块（也就是 `[1]`，和为 1）：`[1, -3]`。和 = $1 + (-3) = -2$ 。
 - 哪个更好，-3 还是 -2？-2 更好（负得少一点）。所以，在这里结束的最佳块是 `[1, -3]`。
 - `max_ending_here` = -2

- 更新 `max_so_far`。新的 `max_ending_here` (-2) 比旧的 `max_so_far` (1) 大吗？不大。
- `max_so_far` 保持为 1
- **移动到第四个数字：4**
 - 在这里结束的最佳块？选择：
 1. 开始新的：[4]。和 = 4。
 2. 扩展前面那个在这里结束的最佳块（也就是 [1, -3]，和为 -2）：[1, -3, 4]。和 = -2 + 4 = 2。
 - 哪个更好，4 还是 2？4 更好。所以，在这里结束的最佳块就是 [4]。
 - `max_ending_here` = 4
 - 更新 `max_so_far`。新的 `max_ending_here` (4) 比旧的 `max_so_far` (1) 大吗？是的。
 - `max_so_far` = 4
- **移动到第五个数字：-1**
 - 在这里结束的最佳块？选择：
 1. 开始新的：[-1]。和 = -1。
 2. 扩展前面那个在这里结束的最佳块（也就是 [4]，和为 4）：[4, -1]。和 = 4 + (-1) = 3。
 - 哪个更好，-1 还是 3？3 更好。所以，在这里结束的最佳块是 [4, -1]。
 - `max_ending_here` = 3
 - 更新 `max_so_far`。3 比 4 大吗？不大。
 - `max_so_far` 保持为 4
- **移动到第六个数字：2**
 - 在这里结束的最佳块？选择：
 1. 开始新的：[2]。和 = 2。
 2. 扩展前面那个在这里结束的最佳块（也就是 [4, -1]，和为 3）：[4, -1, 2]。和 = 3 + 2 = 5。
 - 哪个更好，2 还是 5？5 更好。所以，在这里结束的最佳块是 [4, -1, 2]。
 - `max_ending_here` = 5
 - 更新 `max_so_far`。5 比 4 大吗？是的。
 - `max_so_far` = 5
- **移动到第七个数字：1**
 - 在这里结束的最佳块？选择：
 1. 开始新的：[1]。和 = 1。
 2. 扩展前面那个在这里结束的最佳块（也就是 [4, -1, 2]，和为 5）：[4, -1, 2, 1]。和 = 5 + 1 = 6。
 - 哪个更好，1 还是 6？6 更好。所以，在这里结束的最佳块是 [4, -1, 2, 1]。
 - `max_ending_here` = 6
 - 更新 `max_so_far`。6 比 5 大吗？是的。
 - `max_so_far` = 6
- **移动到第八个数字：-5**
 - 在这里结束的最佳块？选择：
 1. 开始新的：[-5]。和 = -5。
 2. 扩展前面那个在这里结束的最佳块（也就是 [4, -1, 2, 1]，和为 6）：[4, -1, 2, 1, -5]。和 = 6 + (-5) = 1。

- 哪个更好，-5 还是 1？1 更好。所以，在这里结束的最佳块是 `[4, -1, 2, 1, -5]`。
- `max_ending_here` = 1
- 更新 `max_so_far`。1 比 6 大吗？不大。
- `max_so_far` 保持为 6

• 移动到第九个数字：4

- 在这里结束的最佳块？选择：
 1. 开始新的：`[4]`。和 = 4。
 2. 扩展前面那个在这里结束的最佳块（也就是 `[4, -1, 2, 1, -5]`，和为 1）：`[4, -1, 2, 1, -5, 4]`。和 = 1 + 4 = 5。
- 哪个更好，4 还是 5？5 更好。所以，在这里结束的最佳块是 `[4, -1, 2, 1, -5, 4]`。
- `max_ending_here` = 5
- 更新 `max_so_far`。5 比 6 大吗？不大。
- `max_so_far` 保持为 6

列表走完了！我们的最终答案就是存储在 `max_so_far` 里的值，也就是 6。

4. 核心规则（简化版）

你注意到计算 `max_ending_here` 的规律了吗？

当看到当前数字

`num` 时：

```
max_ending_here = max(num, 上一步的_max_ending_here + num)
```

- `max(a, b)` 的意思就是“在 `a` 和 `b` 之间选个大的”。
- 这个规则自动处理了“是开始新块还是扩展旧块”的决定。如果 `上一步的_max_ending_here` 是负数，把它加到 `num` 上只会让结果比 `num` 本身还小，所以 `max` 会选择 `num`（实际上就是开始了新块）。如果它是正数，那么扩展通常更好，`max` 会选择那个和。

并且，我们总是更新全局最优值：

```
max_so_far = max(max_so_far, max_ending_here)
```

5. 为什么这个方法又快又好？

- **速度（时间）：** 我们只把列表从头到尾看了一遍。对于每个数字，我们只做了一次简单的加法和比较。这超级快，叫做 $O(n)$ 时间复杂度——花费的时间和列表的长度 (n) 成正比增长，这是我们能期待的最好的情况了。
- **内存（空间）：** 我们只需要两个额外的变量（`max_so_far` 和 `max_ending_here`）来记录信息，不管列表有多长。这非常节省内存，叫做 $O(1)$ 空间复杂度。

6. 代码实现（和解释对应起来）

```
class Solution {
    public int maxSubArray(int[] nums) {
        // 处理空列表的情况，虽然题目约束说不会发生，但加上更稳健
        if (nums == null || nums.length == 0) {
            return 0;
        }

        // 用第一个数字初始化我们的两个关键变量
        int max_so_far = nums[0]; // 全局最大和 (之前叫 globalMaxSum)
        int max_ending_here = nums[0]; // 以当前位置结尾的最大和 (之前叫 currentMaxSum)

        // 从 *第二个* 数字开始遍历 (索引从 1 开始)
        for (int i = 1; i < nums.length; i++) {
```

```
int current_number = nums[i]; // 当前数字

// 应用核心规则：是扩展之前的最佳结尾块，还是重新开始？
// 在这行执行之前，max_ending_here 保存的是上一步的值
max_ending_here = Math.max(current_number, max_ending_here + current_number);

// 在这里结尾的最佳块，有没有刷新我们目前见过的全局最佳记录？
max_so_far = Math.max(max_so_far, max_ending_here);
}

// 检查完所有数字后，max_so_far 就保存了最终答案
return max_so_far;
}
}
```

这就是全部思路了！通过巧妙地记录“以当前位置结尾的最大和”，我们就能只用一遍扫描就找到全局的最大子数组和。